Aubrée Romain **Beauvallet**
13/11/2026 **Semestre 3**
SLAM

Groupe 1

Bloc 1 TP Git

Contexte **2**
- **Partie 0 : la vie sans Git** **2**
- **Partie 1 : premier dépôt et historique** **3**
- **Partie 2 : incident en production** **5**
- **Partie 3 : branches et fusion** **6**
- **Partie 4 : travail à plusieurs et conflits** **7**
- **Partie 5 : revue de code, hotfix et livraison** **10**
- **Bilan et livrables** **11**

Conclusion **12**

Contexte

Nous formons l'équipe technique de **FleurDeLyon**, un fleuriste lyonnais dont le site vitrine (une page **index.html** et une feuille **style.css**) est modifié chaque semaine par plusieurs personnes. Le but du TP est de vivre, demande après demande, les problèmes d'une petite équipe web (fichiers écrasés, bug en production, travail à plusieurs) et de voir comment **Git** les résout.

Notre équipe compte trois rôles : **Louis** (Dev A, qui a hébergé le dépôt commun sur son compte GitHub ShadowGcraft), **moi, Romain** (Dev B) et **Aymeric** (Responsable livraison). **Mon rôle de Dev B** : en partie 0 je modifie ma copie du site (prix des tulipes et bandeau de livraison) ; aux parties 1 à 3 je travaille seul dans mon propre dépôt ; en partie 4 je clone le dépôt commun, je modifie le style en parallèle des autres puis je provoque le conflit ; en partie 5 je développe la fonctionnalité « offre sur les couronnes » et j'ouvre la pull request.

Dépôt commun de l'équipe : ***GitHub (privé) — https://github.com/ShadowGcraft/fleurdelyon-equipe1****. Chacun a travaillé avec sa propre identité Git, donc* ***git log*** *et* ***git blame*** *attribuent chaque commit à la bonne personne.*

Partie 0 : la vie sans Git

Avant d'utiliser Git, nous reproduisons la méthode encore courante : copier des dossiers et se les envoyer. Chacun copie le dossier de départ. Dans **fleurdelyon_devB**, j'ai passé les tulipes à 20 € et ajouté « Livraison gratuite dès 40 € » sous le titre, pendant que Louis modifiait sa propre copie (roses à 28 €, téléphone).

Aymeric devait réunir nos deux copies au copier-coller. À son premier essai, il a copié le dossier de Louis puis mon fichier par-dessus : **mon travail a survécu, mais celui de Louis a disparu** sans aucun avertissement. Si l'ordre de copie avait été inversé, c'est mon travail qui aurait été perdu : le résultat dépend du hasard. Nous avons relevé **trois problèmes** :

Problème 1 — l'écrasement du travail : la dernière copie posée écrase la précédente. Si deux personnes modifient la même ligne, une des deux versions est forcément perdue, et seul celui qui assemble peut s'en apercevoir.

Problème 2 — l'absence de trace : pour savoir ce qui avait changé, il a fallu comparer les fichiers ligne à ligne. Rien n'indique qui a changé le prix des roses, quand, ni pourquoi.

Problème 3 — l'impossibilité de revenir en arrière : une fois le dossier final assemblé, la version de la veille n'existe plus, sauf si quelqu'un a pensé à en garder une copie.

Partie 1 : premier dépôt et historique

Pour les parties 1 à 3, chacun travaille seul dans son propre dépôt. Git distingue trois zones : le **dossier de travail**, l'**index** (ce qu'on prépare pour le prochain commit) et le **dépôt** (les commits enregistrés) ; **git add** fait passer de la première à la deuxième, **git commit** de la deuxième à la troisième. Comme l'identité Git globale du poste était celle de Louis, j'ai défini la mienne en local (git config sans --global) pour que mes commits soient signés « Romain ».

```
romain@FLEUR-PC MINGW64 ~/fleurdelyon (main)
$ git config user.name "Romain"
romain@FLEUR-PC MINGW64 ~/fleurdelyon (main)
$ git init -b main
Initialized empty Git repository in /c/Users/romain/fleurdelyon/.git/
romain@FLEUR-PC MINGW64 ~/fleurdelyon (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html
        style.css

nothing added to commit but untracked files present (use "git add" to track)
romain@FLEUR-PC MINGW64 ~/fleurdelyon (main)
$ git add index.html style.css
romain@FLEUR-PC MINGW64 ~/fleurdelyon (main)
$ git commit -m "Version initiale du site"
[main (root-commit) 61903ef] Version initiale du site
 2 files changed, 21 insertions(+)
 create mode 100644 index.html
 create mode 100644 style.css
```

Trois commits racontent ensuite l'histoire du site (roses à 28 €, plante verte « en promotion » avec la faute de frappe volontaire à 1 € au lieu de 10 €, puis le bandeau de livraison gratuite). Avant chaque commit, **git diff** montre ce qui va être enregistré :

```
romain@FLEUR-PC MINGW64 ~/fleurdelyon (main)
$ git diff
diff --git a/index.html b/index.html
index e7a6d0f..b72e77c 100644
--- a/index.html
+++ b/index.html
@@ -12,7 +12,7 @@
   <ul>
     <li>Bouquet de roses : 28 €</li>
     <li>Bouquet de tulipes : 18 €</li>
-    <li>Plante verte : 12 €</li>
+    <li>Plante verte : 1 €</li>
   </ul>
   <footer>Contact : 04 00 00 00 00</footer>
 </body>
```

Après un **git add**, **git diff** ne montre plus rien (la modification est passée dans l'index) : il faut alors **git diff --staged** pour la revoir. C'est la différence entre les deux : **git diff** compare le dossier de travail à l'index, **git diff --staged** compare l'index au dernier commit. À la fin, **git log --oneline** affiche les **4 commits**, chacun avec un identifiant court de 7 caractères :

```
romain@FLEUR-PC MINGW64 ~/fleurdelyon (main)
$ git log --oneline
bb3e0a5 Ajoute le bandeau de livraison gratuite
6d00530 Plante verte en promotion
ff91847 Roses à 28 €
45d33a5 Version initiale du site
```

Point de contrôle atteint : 4 commits et **git status** annonce « nothing to commit, working tree clean ».

Et sans Git ? Il aurait fallu quatre copies complètes du site, une par état, dont les noms ne disent ni ce qui a changé, ni qui l'a fait, ni pourquoi (fleurdelyon_v2, fleurdelyon_final, fleurdelyon_final_ok…). Avec Git, un seul dossier suffit : chaque état est un commit avec un identifiant, un auteur, une date et un message.

Partie 2 : incident en production

Un client a commandé vingt plantes vertes à 1 € : la faute de frappe de la partie 1 est devenue un vrai bug, noyé dans un historique qui a avancé depuis. J'enquête avec **git blame**, qui affiche pour chaque ligne le dernier commit qui l'a modifiée, son auteur et sa date. La ligne de la plante verte pointe vers le commit « Plante verte en promotion » :

```
romain@FLEUR-PC MINGW64 ~/fleurdelyon (main)
$ git blame index.html
^45d33a5 (Romain 2026-11-12 09:05:00 +0100  1) <!DOCTYPE html>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100  2) <html lang="fr">
^45d33a5 (Romain 2026-11-12 09:05:00 +0100  3) <head>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100  4)   <meta charset="utf-8">
^45d33a5 (Romain 2026-11-12 09:05:00 +0100  5)   <title>FleurDeLyon</title>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100  6)   <link rel="stylesheet" href="style.css">
^45d33a5 (Romain 2026-11-12 09:05:00 +0100  7) </head>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100  8) <body>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100  9)   <h1>FleurDeLyon</h1>
bb3e0a50 (Romain 2026-11-12 09:50:00 +0100 10)   <p>Livraison gratuite dès 40 €</p>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100 11)   <p>Fleuriste à Lyon depuis 2009.</p>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100 12)   <h2>Nos prix</h2>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100 13)   <ul>
ff918475 (Romain 2026-11-12 09:20:00 +0100 14)     <li>Bouquet de roses : 28 €</li>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100 15)     <li>Bouquet de tulipes : 18 €</li>
6d00530a (Romain 2026-11-12 09:35:00 +0100 16)     <li>Plante verte : 1 €</li>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100 17)   </ul>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100 18)   <footer>Contact : 04 00 00 00 00</footer>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100 19) </body>
^45d33a5 (Romain 2026-11-12 09:05:00 +0100 20) </html>
```

J'annule uniquement ce commit avec **git revert**, qui crée un nouveau commit faisant l'inverse de l'ancien : le bandeau, ajouté après, est conservé. J'applique ensuite le bon prix (10 €). **git log --oneline** affiche maintenant **6 commits** (les 4 d'origine, le Revert et « Plante verte à 10 € ») :

```
romain@FLEUR-PC MINGW64 ~/fleurdelyon (main)
$ git log --oneline
38e69a8 Plante verte à 10 €
cf69808 Revert "Plante verte en promotion"
bb3e0a5 Ajoute le bandeau de livraison gratuite
6d00530 Plante verte en promotion
ff91847 Roses à 28 €
45d33a5 Version initiale du site
```

J'ai aussi testé **git restore** (qui annule une modification non encore committée — attention, elle est alors définitivement perdue) et **git switch --detach** (qui remet le dossier dans l'état d'un ancien commit). On préfère **revert** à **reset --hard** sur un historique partagé : reset réécrit l'historique et ferait disparaître des commits que les collègues ont peut-être déjà récupérés, alors que revert n'ajoute qu'une correction traçable.

Et sans Git ? Sans Git, aucune trace de qui a tapé « 1 € » ni quand : il faudrait interroger chaque collègue de mémoire ou rouvrir les anciennes copies une à une. Pour corriger, il ne resterait qu'à reprendre une vieille copie (et perdre le bandeau) ou corriger à l'aveugle. git blame, git show et git revert donnent la réponse et annulent exactement ce changement, et rien d'autre.

Partie 3 : branches et fusion

Le patron veut une page de Noël, mais le site en ligne (**main**) doit rester corrigeable en urgence. Une **branche** est une ligne de travail parallèle : **git switch -c promo-noel** la crée. Dessus, deux commits : la page **noel.html**, puis un lien « Offres de Noël » dans index.html. Une urgence arrive (le téléphone change) : en revenant sur main, noel.html disparaît du dossier — Git range les fichiers selon la branche choisie, il n'est pas perdu. Je corrige le téléphone sur main, puis je fusionne : comme les deux changements portent sur des lignes différentes, Git les combine seul.

```
romain@FLEUR-PC MINGW64 ~/fleurdelyon (main)
$ git log --oneline --graph
*   9749bed Fusionne la page promo de Noël
|\
| * bda05cd Ajoute le lien vers la page de Noël
| * c82b0ea Ajoute la page promo de Noël
* | 8c2c3ee Corrige le numéro de téléphone
|/
* 38e69a8 Plante verte à 10 €
* cf69808 Revert "Plante verte en promotion"
* bb3e0a5 Ajoute le bandeau de livraison gratuite
* 6d00530 Plante verte en promotion
* ff91847 Roses à 28 €
* 45d33a5 Version initiale du site
```

Point de contrôle atteint : **main** contient la page de Noël et le nouveau téléphone, et le commit de fusion a bien **deux parents** (visible dans le graphe).

Et sans Git ? Il aurait fallu dupliquer le site (site_en_ligne et site_noel), corriger le téléphone d'un côté, puis se souvenir de toutes les modifications de chaque côté pour les reporter à la main. Le moindre oubli aurait remis l'ancien téléphone en ligne. Avec une branche, Git connaît le point de départ commun et fusionne seul quand les lignes diffèrent.

Partie 4 : travail à plusieurs et conflits

Louis a mis son dépôt des parties 1 à 3 sur GitHub (**ShadowGcraft/fleurdelyon-equipe1**). Je l'ai cloné avec **git clone** : j'ai récupéré tout le projet avec son historique, y compris les commits que Louis avait faits seul. J'ai ensuite passé le titre en rouge dans style.css, puis poussé. Comme j'ai poussé le premier, mon **git push** est passé directement ; ce sont Aymeric puis Louis qui ont vu leur push refusé et ont dû faire **git pull** avant de pousser.

Pour provoquer un **conflit**, Louis et moi avons modifié la même ligne (l'accroche de la page d'accueil). J'ai poussé ma version en premier (« Fleuriste artisan à Lyon depuis 2009. ») : c'est donc Louis qui a reçu le conflit au moment de son **git pull** ; **git status** indique « both modified » :

```
louis@FLEUR-PC MINGW64 ~/fleurdelyon-equipe1 (main)
$ git pull
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
From https://github.com/ShadowGcraft/fleurdelyon-equipe1.git
   241efd0..a6d74e9  main       -> origin/main
louis@FLEUR-PC MINGW64 ~/fleurdelyon-equipe1 (main)
$ git status
On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours)

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
```

En ouvrant le fichier, les deux versions apparaissent entre des **marqueurs** : celle de Louis entre **<<<<<<<** et **=======**, la mienne entre **=======** et **>>>>>>>** :

```
louis@FLEUR-PC MINGW64 fleurdelyon-equipe1
$ sed -n "8,17p" index.html
<body>
<h1>FleurDeLyon</h1>
<p>Livraison gratuite dès 40 €</p>
<<<<<<< HEAD
<p>Fleuriste à Lyon depuis 2009, livraison en 2 h.</p>
=======
<p>Fleuriste artisan à Lyon depuis 2009.</p>
>>>>>>> a6d74e9e24eb1d23bedb8a11317ae972a74268e6
<p><a href="noel.html">Offres de Noël</a></p>
```

Nous en avons parlé : les deux informations sont utiles (savoir-faire artisanal et livraison rapide). Nous avons choisi de combiner les deux phrases en « Fleuriste artisan à Lyon depuis 2009, livraison en 2 h. ». Louis a supprimé les marqueurs, enregistré la résolution et poussé ; de mon côté un **git pull** a récupéré le résultat.

git blame donne désormais une réponse nominative — le titre rouge m'est attribué, le README à Aymeric :

```
romain@FLEUR-PC MINGW64 ~/fleurdelyon-equipe1 (main)
$ git blame style.css
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100 1) body { font-family: sans-serif; margin: 2rem; }
86d361de (Romain               2026-11-12 14:00:00 +0100 2) h1 { color: #b71c1c; }
romain@FLEUR-PC MINGW64 ~/fleurdelyon-equipe1 (main)
$ git blame index.html
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100  1) <!DOCTYPE html>
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100  2) <html lang="fr">
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100  3) <head>
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100  4)   <meta charset="utf-8">
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100  5)   <title>FleurDeLyon</title>
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100  6)   <link rel="stylesheet" href="style.css">
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100  7) </head>
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100  8) <body>
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100  9)   <h1>FleurDeLyon</h1>
2ae8ef43 (Herzog Louis Ludovic 2026-11-12 09:50:00 +0100 10)   <p>Livraison gratuite dès 40 €</p>
9f2db549 (Herzog Louis Ludovic 2026-11-12 14:35:00 +0100 11)   <p>Fleuriste artisan à Lyon depuis 2009, livraison en 2 h.</p>
856d634b (Herzog Louis Ludovic 2026-11-12 10:50:00 +0100 12)   <p><a href="noel.html">Offres de Noël</a></p>
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100 13)   <h2>Nos prix</h2>
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100 14)   <ul>
0886024a (Herzog Louis Ludovic 2026-11-12 09:20:00 +0100 15)     <li>Bouquet de roses : 28 €</li>
2d5d7958 (Herzog Louis Ludovic 2026-11-12 15:12:00 +0100 16)     <li>Bouquet de tulipes : 20 €</li>
2323fc9d (Herzog Louis Ludovic 2026-11-12 10:20:00 +0100 17)     <li>Plante verte : 10 €</li>
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100 18)   </ul>
189db041 (Herzog Louis Ludovic 2026-11-12 11:00:00 +0100 19)   <footer>Contact : 04 78 00 00 00</footer>
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100 20) </body>
^321c067 (Herzog Louis Ludovic 2026-11-12 09:05:00 +0100 21) </html>
```

Aucun marqueur **<<<<<<<**, **=======** ou **>>>>>>>** ne reste dans le fichier. À retenir : un conflit n'est pas une erreur, c'est Git qui refuse de deviner entre deux décisions humaines contradictoires, et la seule vraie solution est de se parler.

Et sans Git ? Avec des pièces jointes par mail, le responsable aurait gardé la dernière version ouverte : une des deux phrases aurait été écrasée en silence, sans que personne le sache avant qu'un client ou le patron le remarque. Avec Git, le conflit est détecté et bloquant, les deux versions sont affichées côte à côte, et la décision prise est elle-même enregistrée dans un commit.

Partie 5 : revue de code, hotfix et livraison

À partir d'ici, la règle écrite par Aymeric dans le README s'applique : personne ne pousse sur **main**, tout passe par une branche et une **pull request** relue. Mon rôle : développer la fonctionnalité. Après un git pull, j'ai créé la branche **feature/offre-couronnes**, ajouté « -10 % sur les couronnes » dans noel.html, poussé la branche et ouvert la **pull request n° 1** :

```
romain@FLEUR-PC MINGW64 ~/fleurdelyon-equipe1 (feature/offre-couronnes)
$ git push -u origin feature/offre-couronnes
remote:
remote: Create a pull request for 'feature/offre-couronnes' on GitHub by visiting:
remote:      https://github.com/ShadowGcraft/fleurdelyon-equipe1/pull/new/feature/offre-couronnes
remote:
To https://github.com/ShadowGcraft/fleurdelyon-equipe1.git
 * [new branch]      feature/offre-couronnes -> feature/offre-couronnes
branch 'feature/offre-couronnes' set up to track 'origin/feature/offre-couronnes'.
romain@FLEUR-PC MINGW64 ~/fleurdelyon-equipe1 (feature/offre-couronnes)
$ gh pr create --base main --head feature/offre-couronnes --title "Offre -10 % sur les couronnes"
https://github.com/ShadowGcraft/fleurdelyon-equipe1/pull/1
```

Aymeric a relu les fichiers modifiés et laissé un commentaire sur ma ligne (« Précise la date de fin de l'offre »). J'ai corrigé, commité et poussé sur la même branche : la pull request s'est mise à jour toute seule et contient alors deux commits. Aymeric a ensuite fusionné. En parallèle, Louis a traité une urgence (le prix des tulipes) sur sa propre branche, avec sa propre pull request n° 2 : nos deux travaux ont avancé sans se bloquer. Enfin, Aymeric a posé l'**étiquette (tag) v1.0** sur la version livrée :

```
romain@FLEUR-PC MINGW64 ~/fleurdelyon-equipe1 (main)
$ git tag
v1.0
romain@FLEUR-PC MINGW64 ~/fleurdelyon-equipe1 (main)
$ git show v1.0 --stat --format="%d %s (%an)"
tag v1.0
Tagger: Aymeric <aymeric@example.com>

Version de Noël 2026
 (HEAD -> main, tag: v1.0, origin/main, origin/HEAD) Fusionne l'offre sur les couronnes
```

Point de contrôle atteint : les deux pull requests sont fusionnées (dont une avec un commentaire de relecture), le tag **v1.0** est présent chez les trois membres après un **git pull --tags**, et aucun commit n'est arrivé directement sur main.

Et sans Git ? Sans Git, la relecture se ferait à l'oral ou par mail, sans preuve rattachée à la ligne relue ni à la version livrée. Dans un an, il faudrait espérer qu'une copie « site_noel_definitif » existe encore et corresponde à ce qui a été mis en ligne. La pull request garde tout (code proposé, commentaire, correction, qui a fusionné), et le tag v1.0 pointe pour toujours sur le commit livré.

Bilan et livrables

Le graphe final raconte toute l'histoire du projet : la ligne droite des parties 1 et 2, la branche de Noël (partie 3), les fusions dues au travail en parallèle et au conflit (partie 4), puis les deux pull requests (partie 5) :

```
romain@FLEUR-PC MINGW64 ~/fleurdelyon-equipe1 (main)
$ git log --oneline --graph
*   ce4678a Fusionne l'offre sur les couronnes
|\
| * 45d2c8e Précise la date de fin de l'offre
| * 16a2bfe Ajoute l'offre sur les couronnes
* |   872067d Fusionne le correctif du prix des tulipes
|\ \
| |/
|/|
| * 2d5d795 Corrige le prix des tulipes
|/
*   9f2db54 Merge branch 'main' of https://github.com/ShadowGcraft/fleurdelyon-equipe1.git
|\
| * 3fc712b Met en avant le savoir-faire artisanal
* | 61dd3e6 Annonce la livraison en 2 h
|/
*   b4a3db3 Merge branch 'main' of https://github.com/ShadowGcraft/fleurdelyon-equipe1.git
|\
| *   fb0326e Merge branch 'main' of https://github.com/ShadowGcraft/fleurdelyon-equipe1.git
| |\
| | * 86d361d Passe le titre en rouge
| * | 9a64623 Ajoute les règles de l'équipe
| |/
* / e1d185a Précise la date limite de commande de Noël
|/
*   bb4daf6 Fusionne la page promo de Noël
|\
| * 856d634 Ajoute le lien vers la page de Noël
| * 8ac1cf5 Ajoute la page promo de Noël
* | 189db04 Corrige le numéro de téléphone
|/
* 2323fc9 Plante verte à 10 €
* 5c2f2ca Revert "Plante verte en promotion"
* 2ae8ef4 Ajoute le bandeau de livraison gratuite
* 851f875 Plante verte en promotion
* 0886024 Roses à 28 €
* 321c067 Version initiale du site
```

Livrables : dépôt de l'équipe **ShadowGcraft/fleurdelyon-equipe1** (privé) ; pull request relue (offre sur les couronnes, moi) : **.../pull/1** ; pull request du correctif urgent (prix des tulipes, Louis) : **.../pull/2** ; version livrée : **tree/v1.0**.

Le moment où Git a été le plus utile, pour moi, est l'incident de la partie 2 : retrouver en une commande le commit fautif, son auteur et sa date, puis l'annuler

sans perdre le bandeau ajouté ensuite, est impossible proprement avec des copies de dossiers. À l'inverse, Git sert peu pour des fichiers binaires volumineux (vidéos, .psd) qu'il ne sait ni comparer ni fusionner, ou pour un document d'une page écrit seul. On pourrait aussi l'utiliser hors du code : un rapport de stage, les fichiers de configuration d'un serveur, la documentation d'une équipe.

Conclusion

Toutes les demandes du patron ont été traitées avec **Git** : un historique de chaque changement, un bug de prix retrouvé par **git blame** et annulé par **git revert**, une page de Noël préparée sur une branche, un travail à trois sur un dépôt GitHub avec un conflit résolu par une décision commune, et ma fonctionnalité livrée après relecture dans la version **v1.0**. Chacun des trois problèmes de la partie 0 (écrasement, absence de trace, impossibilité de revenir en arrière) a trouvé sa réponse.

Principales **difficultés rencontrées** : bien distinguer les trois zones de Git (dossier de travail, index, dépôt), que **git diff** et **git diff --staged** rendent visibles ; penser à régler **git config pull.rebase false** pour obtenir une fusion (avec marqueurs) au lieu d'un rebase ; et surtout, comme toute l'équipe poussait avec le compte GitHub de Louis, ma pull request était « la sienne » pour GitHub : Aymeric n'a pas pu cliquer formellement sur « Approve » et a donc fait sa relecture en commentaires. Avec un compte par personne, la relecture aurait pu être approuvée officiellement.